

claude-windows / e83b09d8-e113-4d4d-9411-cd80ae217af9

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\subagents\workflows\wf_ae339437-da9\agent-a9aaabf6f747c1caf.jsonl`  
- SHA-256: `901cb9f95cbe3406f3d062aae692cf5d01f2454f869e8ada07f5ff070bd9a452`  
- Source modified: `2026-07-07T15:52:19+00:00`  
- Imported at: `2026-07-08T15:59:35+00:00`  
- project: `wf_ae339437-da9`  
- session_id: `e83b09d8-e113-4d4d-9411-cd80ae217af9`
```

Transcript

1. user (2026-07-07T15:48:13.159Z)

You are reviewing ONE commit on branch `fix/cloud-run-exec-status-fastfail` in the repo at:

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
Always start by: `cd "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer"` (the Bash cwd may default elsewhere). Use Read/Grep with absolute paths under that repo.

The commit hardens `CloudRunCompute.run\_infer` (backend/compute/cloud_run.py): the done-marker stays the
SOLE success signal, but each bucket-poll ALSO probes the Cloud Run Executions API so a worker that
TERMINATED without ever writing a marker (crash/OOM/an old image argparse-aborting on an unknown flag)
fails fast (raise RemoteExecutionFailed) instead of polling the full remote_poll_max_wait_sec (~65 min).
`\_dispatch\_remote` maps RemoteExecutionFailed -> rc 5 (distinct from the rc 4 poll-timeout).

CRITICAL CONTEXT: this commit was REBASED onto a master that had since merged two sibling PRs touching
the same files:
- #513 (23c1868): added `ComputeJobSpec.skip\_validation` + a `--skip-validation` worker flag;
the

control-plane dispatcher sets it so the worker skips redundant re-validation.
- #512 (20862d7): a validators shared-decode refactor (file-disjoint from this commit).
The motivating scenario for THIS fix: a post-#513 control-plane dispatching `--skip-validation`
to an
OLD worker image (no such flag) -> the worker argparse-aborts (exit 2) BEFORE writing a marker
-> the
exact ~65-min hang this fix collapses to a prompt rc-5 failure.

To see EXACTLY what this commit changes (only 5 files), run:

```
git diff origin/master...HEAD      (three-dot: my change vs merge-base)  
To see the sibling context that landed on the same files, run:  
git diff 3be0935..origin/master -- backend/compute/base.py backend/compute/cloud_run.py  
backend/worker/__main__.py
```

Read the FULL current files (not just the diff) to judge correctness in context:

```
backend/compute/cloud_run.py, backend/compute/base.py, backend/compute/__init__.py,  
backend/worker/__main__.py, backend/orchestration.py (the API caller),  
tests/test_compute_backend.py,  
backend/infrastructure/execution_policy.py (poll_until semantics).
```

Report ONLY real, defect-grade findings with a concrete failure scenario. Do NOT report style nits,
things the gate already covers, or speculation. If the change is correct on your lens, return an

empty
findings array and say so in `assessment`. Full suite (1833) + ruff + mypy + coverage 85.66%
already pass;
do not re-report "add a test" unless a SPECIFIC untested behavior would plausibly break.

LENS ? TEST ADEQUACY. Read tests/test_compute_backend.py in full. Verify the new tests genuinely exercise the fast-fail (not tautological), and specifically whether the MOTIVATING end-to-end scenario is covered: dispatcher sets skip_validation -> worker argparse-aborts pre-marker -> execution terminal -> RemoteExecutionFailed -> rc 5. Identify at most ONE high-value MISSING test whose absence leaves a plausible real regression uncaught (e.g. an integration test tying skip_validation dispatch to the rc-5 fast-fail). Do not pad; if coverage is adequate, say so.

2. assistant (2026-07-07T15:48:15.201Z)

I'll start by navigating to the repo and examining the commit under review.

3. user (2026-07-07T15:48:18.721Z)

```
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
---BRANCH---
fix/cloud-run-exec-status-fastfail
---DIFF vs merge-base---
 backend/compute/__init__.py | 8 +-
 backend/compute/base.py      | 14 +++
 backend/compute/cloud_run.py | 115 ++++++
 backend/worker/__main__.py   | 16 +-
 tests/test_compute_backend.py | 229 ++++++
5 files changed, 375 insertions(+), 7 deletions(-)
```

4. user (2026-07-07T15:48:19.522Z)

```
diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
index 1091582..482f3c9 100644
--- a/backend/compute/__init__.py
+++ b/backend/compute/__init__.py
@@ -13,13 +13,19 @@ import dataclasses
 import os
 from typing import Any, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,
+    RemoteExecutionFailed,
+)
 from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

__all__ = [
    "ComputeBackend",
    "ComputeJobSpec",
    "ComputeResult",
+    "RemoteExecutionFailed",
    "get_compute_backend",
]

diff --git a/backend/compute/base.py b/backend/compute/base.py
index 31af3fe..3ad76fc 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
```

```

@@ -17,6 +17,20 @@ from dataclasses import dataclass, field
    from typing import Callable, Optional

+class RemoteExecutionFailed(RuntimeError):
+    """A dispatched remote job's execution reached a TERMINAL state without ever writing its
+    completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image argparse-
    rejecting
+    an unknown flag) before the marker push, or the marker upload itself failed on an otherwise
+    Succeeded run.
+
+    Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL be
+    running): this says the execution is DEFINITELY over and no marker will ever appear, so
    the
+    dispatcher must stop bucket-polling immediately instead of burning the full
+    ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message carries the
+    job + execution name + terminal state + the ``gcloud`` command to read the execution log,
    so
+    the whole "worker crashed before the done-marker" class fails fast AND diagnostically."""
+
+
+    @dataclass(frozen=True)
+    class ComputeJobSpec:
+        """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will (re)compute
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 7247ace..1e6e313 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -11,6 +11,10 @@ Flow (the cloud-serving compute path):
    job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-
    healthy
    worker that finished just past the deadline is rescued, not wasted ? then best-effort
    CANCELS
    the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
+    *Fast negative path:* while the marker stays the SOLE success signal, each poll also
    probes the
+    Cloud Run Executions API ? if the execution has TERMINATED without ever writing a marker
    (a
+    worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
    diagnostically
+    (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile wait.
    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

@@ -32,7 +36,12 @@ import uuid
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,
+    RemoteExecutionFailed,
+)
    from backend.infrastructure.execution_policy import (
        DEFAULT_POLICY,
        ExecutionPolicy,
@@ -76,6 +85,21 @@ class CloudRunJobClient(ABC):
    May raise ? the caller treats every failure as best-effort (the original poll timeout
    is
        NEVER masked by a cancel error)."""

+    def execution_terminal_state(self, execution: str) -> Optional[str]:
+        """Return the execution's TERMINAL state as a short label (``"Failed"`` /

```

```

`"Cancelled"` /
+     `"Succeeded"` if it has finished, else `None` (still pending/running, or the state
+     can't be determined).
+
+     ADVISORY, not authoritative: it feeds `run_infer`'s fast negative path so a worker
that
+     terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails fast
instead
+     of bucket-polling the full budget. The done-marker stays the ONLY success signal.
+
+     NON-abstract with a default of `None` (additive / default-OFF): a client that can't ?
or
+     chooses not to ? poll the Executions API degrades transparently to today's marker-only
+     polling. Implementations MAY raise on a transient API error ? `run_infer` treats a
raise
+     as `None` (unknown ? keep waiting), so a flaky status API never fails a healthy
run. """
+     return None
+

class GoogleCloudRunJobClient(CloudRunJobClient):
    """Real client ? lazy-imports `google-cloud-run`. Owner-verified on the M7 real run; CI
uses
@@ -145,6 +169,35 @@ class GoogleCloudRunJobClient(CloudRunJobClient):
    ) # pragma: no cover - exercised only on a real run
    client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))

+ def execution_terminal_state(self, execution: str) -> Optional[str]:
+     # Advisory status probe (see the ABC docstring): describe the execution and, ONLY once
it
+     # is terminal (`completion_time` set), classify it. A non-resolvable name / absent
SDK
+     # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
+     # raises on bad input, because the probe is best-effort and must not fail a healthy
run.
+     if "/executions/" not in (execution or ""):
+         return None
+     try:
+         from google.cloud import run_v2 # type: ignore[attr-defined]
+     except Exception: # pragma: no cover - real SDK not present in CI
+         return None
+     client = (
+         run_v2.ExecutionsClient()
+     ) # pragma: no cover - exercised only on a real run
+     exe = client.get_execution( # pragma: no cover - real run only
+         request=run_v2.GetExecutionRequest(name=execution)
+     )
+     # completion_time is a proto Timestamp: unset ? seconds==0 ? still running/pending.
+     if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
+         return None # pragma: no cover - real run only
+     # Terminal: prefer the explicit per-task counts (a cancel shows as cancelled_count>0).
+     if getattr(exe, "cancelled_count", 0): # pragma: no cover - real run only
+         return "Cancelled"
+     if getattr(exe, "failed_count", 0): # pragma: no cover - real run only
+         return "Failed"
+     if getattr(exe, "succeeded_count", 0): # pragma: no cover - real run only
+         return "Succeeded"
+     return "Failed" # pragma: no cover - terminal but no positive success ? treat as
failed
+

class CloudRunCompute(ComputeBackend):
    """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
@@ -215,7 +268,7 @@ class CloudRunCompute(ComputeBackend):

```

```

try:
    marker = poll_until(
-         lambda: self._read_marker(done_uri),
+         lambda: self._poll_check(done_uri, str(execution or "")),
        self._policy,
        label="cloud-run-infer",
        logger=LOG,
@@ -224,6 +277,9 @@ class CloudRunCompute(ComputeBackend):
        on_tick=on_wait,
    )
except PolicyTimeout as timeout_exc:
+     # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
+     # so it propagates past here uncaught ? the dispatch handler maps it to a distinct,
+     # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
        marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
        video_id = str(marker.get("video_id", ""))
        # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
@@ -249,6 +305,61 @@ class CloudRunCompute(ComputeBackend):
        execution=str(execution or ""),
    )

+ def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
+     """One bucket-poll iteration, hardened with an Executions-API fast negative path.
+
+     The done-marker is the SOLE success signal and is checked FIRST ? a present marker
always
+     wins (even if the execution's status later flips), so a worker that wrote a SUCCESS *or
a
+     FAILURE* marker resolves through the normal path with its real returncode. Only when
the
+     marker is absent do we consult the execution status: a worker that TERMINATED without
ever
+     writing a marker (crash / OOM / an old image argparse-aborting on an unknown flag)
would
+     otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min) before
failing
+     opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
+     failure.
+
+     Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
+     ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over, no
marker
+     will ever appear)."""
+     marker = self._read_marker(done_uri)
+     if marker is not None:
+         return marker # success signal ? authoritative even if status later flips
+     state = self._terminal_execution_state(execution)
+     if state is None:
+         return None # still pending/running, or status unknown ? keep bucket-polling
+     # The execution is terminal but no marker is present. Re-check the marker ONCE to close
the
+     # race where the worker wrote it between our two reads (mirrors the post-timeout rescue
in
+     # _handle_poll_timeout). If it is genuinely absent, the process is gone and no marker
will
+     # EVER appear ? fail fast + diagnostically rather than wait out the whole poll budget.
+     marker = self._read_marker(done_uri)
+     if marker is not None:
+         return marker
+     raise RemoteExecutionFailed(
+         f"cloud-run: job={self._job_name} execution={execution or '<unknown>'} reached "

```

```

+         f"terminal state {state!r} with NO completion marker ? the worker exited before "
+         "writing one (crash / OOM / pre-marker abort, or a failed marker upload on an "
+         "otherwise-Succeeded run). The execution is over, so the marker will never appear;
+
+         "inspect the Cloud Run Job execution log to diagnose: "
+         f"`gcloud run jobs executions describe {execution or '<execution>'} "
+         f"--region {self._region}`."
+     )
+
+ def _terminal_execution_state(self, execution: str) -> Optional[str]:
+     """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A raising
or
+     flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
+     so any exception ? and an empty/unresolvable execution name ? degrades to ``None``
(unknown
+     ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
+     if not execution:
+         return None
+     try:
+         return self._client.execution_terminal_state(execution)
+     except Exception: # noqa: BLE001 ? advisory probe: never let it kill the bucket-poll
+         LOG.debug(
+             "cloud-run: execution status probe failed for %s ? treating as unknown",
+             execution,
+             exc_info=True,
+         )
+         return None
+
+ def _handle_poll_timeout(
+     self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
+ ) -> dict:
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 9b19215..6d3420b 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -176,9 +176,14 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
    container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.

- A remote job that never writes a completion marker (a failed/hung container) makes the
bucket
- poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote
dispatch
- timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
+ A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
+ from the local 2/3), never a raw traceback:
+ * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution may
still be
+ running (the shim best-effort cancels it).
+ * rc 5 ? the execution has TERMINATED without a marker (``RemoteExecutionFailed``): a
worker
+ crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead of the
+ ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
+ from backend.compute import RemoteExecutionFailed
+ from backend.infrastructure.execution_policy import PolicyTimeout

    spec = ComputeJobSpec(
@@ -190,6 +195,11 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    )
    try:

```

```

        result = compute.run_infer(spec)
+     except RemoteExecutionFailed as e:
+         logger.error(
+             "[worker] remote execution terminated without a completion marker: %s", e
+         )
+         return 5
    except PolicyTimeout as e:
        logger.warning(
            "[worker] remote compute did not complete (no marker within budget): %s", e
diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py
index faa4981..07e8068 100644
--- a/tests/test_compute_backend.py
+++ b/tests/test_compute_backend.py
@@ -12,7 +12,11 @@ import tempfile

import pytest

-from backend.compute import ComputeJobSpec, get_compute_backend
+from backend.compute import (
+    ComputeJobSpec,
+    RemoteExecutionFailed,
+    get_compute_backend,
+)
from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
from backend.config import AppConfig
from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout
@@ -110,6 +114,37 @@ class _NeverDoneClient(CloudRunJobClient):
    raise self._cancel_error

+class _CrashedNoMarkerClient(CloudRunJobClient):
+    """Simulates the bug's failure class: the worker execution TERMINATES (crash / OOM / an old
+    image argparse-aborting on an unknown flag) WITHOUT ever writing a done-marker. Dispatch
+    returns
+    a resolvable execution name, no marker is ever written, and the Executions API reports a
+    terminal
+    state ? so run_infer must fail FAST via the status probe instead of spinning the poll
+    budget."""
+
+    EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-crash"
+
+    def __init__(self, *, state: str = "Failed"):
+        self.calls: list[dict] = []
+        self.cancelled: list[str] = []
+        self.status_polls = 0
+        self._state = state
+
+    def run_job(self, job_name, argv, *, region, project=None):
+        self.calls.append(
+            {"job": job_name, "argv": list(argv), "region": region, "project": project}
+        )
+        return self.EXEC_NAME
+
+    def cancel_execution(self, execution):
+        self.cancelled.append(execution)
+
+    def execution_terminal_state(self, execution):
+        self.status_polls += 1
+        # Anti-hang guard: the fast path MUST abort on the first terminal observation. If a
+        # regression kept polling despite a terminal status, trip here instead of spinning.
+        assert self.status_polls <= 5, "status probed repeatedly ? fast path did not abort the
poll"
+        return self._state
+
+

```

```

# ----- factory (default-OFF)

@@ -423,6 +458,198 @@ def test_poll_timeout_late_marker_rescues_result(monkeypatch):
    assert exists_calls["n"] == 2 # exactly ONE extra post-timeout check

+# ----- execution-status fast negative path (worker crashed pre-
marker)
+
+# poll_every_n_sec=0.0 ? the poll spins as fast as possible; max_wait_sec=2.0 BOUNDS any
regression
+# (a fast path that silently fell back to marker-only polling fails in ~2s here, never a 65-min
hang)
+# while the working fast path aborts on poll 0 ? instantly.
+_FAILFAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=2.0)
+
+
+def test_run_infer_fails_fast_when_execution_terminal_without_marker(tmp_path):
+    """The core fix: a worker execution that TERMINATED (Failed) without ever writing a marker
is
+    caught by the Executions-API probe ? RemoteExecutionFailed FAST (poll 0), not a ~65-min
opaque
+    hang. The diagnostic names the execution, the terminal state, and the log-inspection
command."""
+    client = _CrashedNoMarkerClient(state="Failed")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )
+    with pytest.raises(RemoteExecutionFailed) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    msg = str(exc.value)
+    assert _CrashedNoMarkerClient.EXEC_NAME in msg # operator can find the execution
+    assert "Failed" in msg # the terminal state is surfaced
+    assert "gcloud run jobs executions describe" in msg # actionable log pointer
+    assert client.status_polls == 1 # aborted on the FIRST terminal observation (fast, not
spun)
+    assert client.cancelled == [] # a finished execution is never cancelled (nothing to
cancel)
+
+
+def test_dispatch_remote_maps_terminal_without_marker_to_rc5(tmp_path):
+    """The dispatch handler maps RemoteExecutionFailed to a distinct, clearly-labelled rc 5 ?
NOT
+    the rc 4 poll-timeout path (a still-maybe-running execution) and NOT a raw traceback ? so
+    'worker crashed before the marker' is operationally distinguishable and surfaces
immediately."""
+    import argparse
+
+    from backend.worker import __main__ as wmain
+
+    client = _CrashedNoMarkerClient(state="Failed")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )

```



```

+     args = argparse.Namespace(
+         video_uri="gs://b/v.mp4",
+         out_uri=str(tmp_path / "bucket"),
+         segmenter="wasb_hybrid",
+         set=None,
+     )
+     rc = wmain._dispatch_remote(backend, args)
+     assert rc == 5 # the crashed-pre-marker fast-fail code...
+     assert rc != 4 # ...distinct from the poll-budget timeout
+     assert client.status_polls == 1 # reached the fast path (not the timeout branch)
+
+
+def test_succeeded_state_without_marker_also_fails_fast(tmp_path):
+    """Generalization: ANY terminal state with no marker is a guaranteed-futile wait. A
+    'Succeeded'
+    execution that nonetheless wrote no marker (the marker upload itself failed) also fails
+    fast,
+    with the state surfaced ? the whole 'execution over, no marker' class collapses, not just
+    Failed."""
+    client = _CrashedNoMarkerClient(state="Succeeded")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )
+    with pytest.raises(RemoteExecutionFailed) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert "Succeeded" in str(exc.value)
+
+
+def test_present_marker_wins_over_terminal_status(tmp_path):
+    """The done-marker stays AUTHORITATIVE: a worker that wrote a marker ? even a FAILURE
+    marker
+    (rc 2) ? resolves through the normal path with its REAL returncode, never a synthesized
+    RemoteExecutionFailed, even when the execution status also reads terminal. Status is a
+    negative
+    path only; it must never override a real marker (which is checked first)."""
+
+    class _MarkerAndFailedStatus(_FakeRunClient):
+        def execution_terminal_state(self, execution): # pragma: no cover - never reached
+            return "Failed" # the marker resolves poll 0, so this is never consulted
+
+    client = _MarkerAndFailedStatus(video_id="abc", returncode=2)
+    backend = CloudRunCompute(
+        run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
+    )
+    result = backend.run_infer(
+        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+    )
+    assert result.returncode == 2 # the worker's REAL rc, not RemoteExecutionFailed
+    assert result.video_id == "abc"
+
+
+def test_status_probe_error_is_swallowed_and_falls_back_to_marker_polling(tmp_path):
+    """Best-effort probe: a raising Executions API must NEVER fail a healthy run. The wait
+    degrades
+    to today's marker-only polling ? PolicyTimeout (the existing timeout path), NOT
+    RemoteExecutionFailed ? a flaky status API cannot manufacture a failure."""
+
+    class _RaisingStatusClient(_NeverDoneClient):
+        def execution_terminal_state(self, execution):

```

```

+         raise RuntimeError("executions API unreachable")
+
+     client = _RaisingStatusClient()
+     backend = CloudRunCompute(
+         run_client=client,
+         job_name="rally-worker",
+         region="asia-south1",
+         policy=_TIMEOUT_NOW,
+         token="tok",
+     )
+     with pytest.raises(PolicyTimeout): # swallowed probe error ? the unchanged timeout path
+         backend.run_infer(
+             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+         )
+     assert client.cancelled == [_NeverDoneClient.EXEC_NAME] # still cancels on the real
timeout
+
+
+def test_default_execution_terminal_state_is_none():
+     """The ABC default is None (additive / default-OFF): a client that doesn't override status
+     polling degrades transparently to marker-only behaviour, so every pre-existing fake is
+     unaffected and the seam never changes a healthy run's outcome."""
+     assert _NoopRunClient().execution_terminal_state("whatever") is None
+     assert (
+         _NeverDoneClient().execution_terminal_state(_NeverDoneClient.EXEC_NAME) is None
+     )
+
+
+def test_real_client_terminal_state_none_for_unresolvable_name():
+     """The real client's status probe is best-effort: a non-resolvable execution name returns
None
+     (can't probe ? keep marker-polling), NOT a raise ? exercisable in CI (the SDK is
absent)."""
+     from backend.compute.cloud_run import GoogleCloudRunJobClient
+
+     client = GoogleCloudRunJobClient()
+     for bogus in ("", "execution", "exec-123"):
+         assert client.execution_terminal_state(bogus) is None
+
+
+def test_terminal_status_with_marker_written_in_race_resolves_via_marker(monkeypatch):
+     """Concurrency race: the execution reads terminal exactly as the worker finishes writing
its
+     marker. _poll_check re-checks the marker ONCE after a terminal status ? if it landed in
between,
+     the completed result is ingested (the marker wins), NOT a spurious
RemoteExecutionFailed."""
+     from backend.compute import cloud_run as cr
+
+     exists_calls = {"n": 0}
+
+     class _FakeStore:
+         def exists(self, ref):
+             exists_calls["n"] += 1
+             # absent on the pre-status read, present on the ONE post-status race re-check
+             return exists_calls["n"] >= 2
+
+     monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
+
+     def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+         if dst is None:
+             dst = os.path.join(tempfile.mkdtemp(prefix="cr-race-"), "m.json")
+         with open(dst, "w", encoding="utf-8") as f:
+             f.write({'"video_id": "vrace", "returncode": 0, "segment_count": 1}')
+         return dst

```

```

+
+ monkeypatch.setattr(cr, "fetch", fake_fetch)
+ client = _CrashedNoMarkerClient(state="Failed") # status says terminal-Failed...
+ backend = cr.CloudRunCompute(
+     run_client=client, job_name="j", region="r", token="t", policy=_FAILFAST
+ )
+ result = backend.run_infer(
+     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
+ )
+ assert result.video_id == "vrace" # ...but the marker that raced in WINS
+ assert exists_calls["n"] == 2 # exactly the pre-status read + the ONE race re-check
+ assert client.status_polls == 1 # status was consulted once, then the marker superseded it
+
+
+def test_terminal_execution_state_none_for_empty_execution():
+    """An empty/unknown execution name (run_job could not resolve one) can't be probed ? None
+    (degrade to marker-only polling), and the client's status API is never even called."""
+
+    class _Boom(_NoopRunClient):
+        def execution_terminal_state(self, execution): # pragma: no cover - must never be
called
+            raise AssertionError("must not probe an empty execution name")
+
+    backend = CloudRunCompute(run_client=_Boom(), job_name="j", region="r")
+    assert backend._terminal_execution_state("") is None
+
+
+    # ----- worker wiring: dispatch branch + done-
marker

```

5. assistant (2026-07-07T15:48:23.107Z)

Let me read the full current files to judge test adequacy in context.

6. user (2026-07-07T15:48:24.978Z)

```

1     """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3     All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4     worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5     ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6     selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7     """
8
9     import json
10    import os
11    import tempfile
12
13    import pytest
14
15    from backend.compute import (
16        ComputeJobSpec,
17        RemoteExecutionFailed,
18        get_compute_backend,
19    )
20    from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
21    from backend.config import AppConfig
22    from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout
23
24    # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).

```

```

25     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
26
27
28     class _FakeRunClient(CloudRunJobClient):
29         """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
local bucket
30         exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""
31
32         def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
33             self.calls: list[dict] = []
34             self.video_id, self.segment_count, self.returncode = (
35                 video_id,
36                 segment_count,
37                 returncode,
38             )
39
40         def run_job(self, job_name, argv, *, region, project=None):
41             self.calls.append(
42                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
43             )
44             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
45             done_uri = argv[argv.index("--done-uri") + 1]
46             outputs = os.path.join(out_uri, "outputs", self.video_id)
47             os.makedirs(outputs, exist_ok=True)
48             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
49                 json.dump(
50                     {
51                         "video_id": self.video_id,
52                         "segment_count": self.segment_count,
53                         "status": "success",
54                     },
55                     f,
56                 )
57             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
58             with open(done_uri, "w", encoding="utf-8") as f:
59                 json.dump(
60                     {
61                         "video_id": self.video_id,
62                         "returncode": self.returncode,
63                         "segment_count": self.segment_count,
64                         "status": "success",
65                     },
66                     f,
67                 )
68             return "exec-123"
69
70         def cancel_execution(self, execution):
71             raise AssertionError(
72                 "cancel must never fire on the happy path"
73             ) # pragma: no cover
74
75
76     class _NoopRunClient(CloudRunJobClient):
77         """Records the dispatch but does NO filesystem writes ? for tests that fake the
storage layer
78         directly (a gs:// out_uri has no local dir to makedirs)."""
79
80         def __init__(self):
81             self.calls: list[dict] = []
82             self.cancelled: list[str] = []
83
84         def run_job(self, job_name, argv, *, region, project=None):
85             self.calls.append(
86                 {"job": job_name, "argv": list(argv), "region": region, "project": project}

```

```

87         )
88         return "exec"
89
90     def cancel_execution(self, execution):
91         self.cancelled.append(execution)
92
93
94     class _NeverDoneClient(CloudRunJobClient):
95         """Simulates a still-running execution: dispatch succeeds (returning a fully-
qualified
96         execution name) but NO done-marker is ever written, so the bucket-poll must time
out."""
97
98         EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-
slow"
99
100        def __init__(self, *, cancel_error: Exception | None = None):
101            self.calls: list[dict] = []
102            self.cancelled: list[str] = []
103            self._cancel_error = cancel_error
104
105        def run_job(self, job_name, argv, *, region, project=None):
106            self.calls.append(
107                {"job": job_name, "argv": list(argv), "region": region, "project": project}
108            )
109            return self.EXEC_NAME
110
111        def cancel_execution(self, execution):
112            self.cancelled.append(execution)
113            if self._cancel_error is not None:
114                raise self._cancel_error
115
116
117        class _CrashedNoMarkerClient(CloudRunJobClient):
118            """Simulates the bug's failure class: the worker execution TERMINATES (crash / OOM /
an old
119            image argparse-aborting on an unknown flag) WITHOUT ever writing a done-marker.
Dispatch returns
120            a resolvable execution name, no marker is ever written, and the Executions API
reports a terminal
121            state ? so run_infer must fail FAST via the status probe instead of spinning the
poll budget."""
122
123            EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-
crash"
124
125            def __init__(self, *, state: str = "Failed"):
126                self.calls: list[dict] = []
127                self.cancelled: list[str] = []
128                self.status_polls = 0
129                self._state = state
130
131            def run_job(self, job_name, argv, *, region, project=None):
132                self.calls.append(
133                    {"job": job_name, "argv": list(argv), "region": region, "project": project}
134                )
135                return self.EXEC_NAME
136
137            def cancel_execution(self, execution):
138                self.cancelled.append(execution)
139
140            def execution_terminal_state(self, execution):
141                self.status_polls += 1
142                # Anti-hang guard: the fast path MUST abort on the first terminal observation.
If a

```

```

143         # regression kept polling despite a terminal status, trip here instead of
spinning.
144         assert self.status_polls <= 5, "status probed repeatedly ? fast path did not
abort the poll"
145         return self._state
146
147
148     # ----- factory (default-
OFF)
149
150
151     def _cfg(compute_target="", storage_backend="local"):
152         return AppConfig.model_validate(
153             {
154                 "deployment": {"compute_target": compute_target},
155                 "storage": {"backend": storage_backend},
156             }
157         )
158
159
160     def test_factory_is_none_for_local_targets():
161         """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
162         assert get_compute_backend(_cfg("")) is None
163         assert get_compute_backend(_cfg("local_gpu")) is None
164         assert get_compute_backend(_cfg("cpu_mock")) is None
165
166
167     def test_factory_returns_cloud_run_for_cloud_target():
168         backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
169         assert isinstance(backend, CloudRunCompute)
170
171
172     # ----- real client: missing-project
guard
173
174
175     def test_real_client_rejects_missing_project():
176         """The real GoogleCloudRunJobClient must fail loudly when GOOGLE_CLOUD_PROJECT is
unset ?
177         a None/empty project would build a bogus "projects/None/..." job path (job_path
requires a
178         real str). The guard runs BEFORE the google-cloud-run import, so it is exercisable
in CI
179         (the SDK is intentionally not a test dependency)."""
180         from backend.compute.cloud_run import GoogleCloudRunJobClient
181
182         client = GoogleCloudRunJobClient()
183         for missing in (None, ""):
184             with pytest.raises(RuntimeError, match="GOOGLE_CLOUD_PROJECT"):
185                 client.run_job(
186                     "rally-worker", ["infer"], region="asia-south1", project=missing
187                 )
188
189
190     def test_real_client_cancel_rejects_unresolvable_execution_name():
191         """cancel_execution must fail loudly on a non-resolvable execution name ? run_job
falls back
192         to the literal "execution" when the operation metadata carries no name, and
cancelling that
193         would be a bogus API call. The guard runs BEFORE the google-cloud-run import
(mirrors the
194         project guard), so it is exercisable in CI (the SDK is intentionally not a test
dependency)."""
195         from backend.compute.cloud_run import GoogleCloudRunJobClient

```

```

196
197     client = GoogleCloudRunJobClient()
198     for bogus in ("", "execution", "exec-123"):
199         with pytest.raises(RuntimeError, match="fully-qualified"):
200             client.cancel_execution(bogus)
201
202
203     # ----- CloudRunCompute dispatch
+ poll
204
205
206     def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
207         client = _FakeRunClient(video_id="abc123", segment_count=3)
208         backend = CloudRunCompute(
209             run_client=client,
210             job_name="rally-worker",
211             region="asia-south1",
212             project="proj",
213             storage_config={},
214             policy=_FAST,
215             token="tok",
216         )
217         out = str(tmp_path / "bucket")
218         result = backend.run_infer(
219             ComputeJobSpec(
220                 video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"
221             )
222         )
223
224         assert result.returncode == 0
225         assert result.video_id == "abc123"
226         assert result.outputs_uri == f"{out}/outputs/abc123/"
227         assert result.report.get("segment_count") == 3
228         # exactly one Cloud Run Job execution was triggered, in the right region/project
229         assert len(client.calls) == 1
230         assert (
231             client.calls[0]["job"] == "rally-worker"
232             and client.calls[0]["region"] == "asia-south1"
233         )
234
235
236     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
237         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
238         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
239         client = _FakeRunClient()
240         backend = CloudRunCompute(
241             run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
242         )
243         backend.run_infer(
244             ComputeJobSpec(
245                 video_uri="gs://b/v.mp4",
246                 out_uri=str(tmp_path / "bk"),
247                 segmenter="wasb_hybrid",
248                 config_overrides={"sport": "tennis"},
249             )
250         )
251
252         argv = client.calls[0]["argv"]
253         assert argv[0] == "infer"
254         assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
255         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
256         # A DEFAULT spec (skip_validation=False) must NOT carry --skip-validation: the
worker still

```

```

257         # validates unless the dispatcher explicitly opts out. Guards the `if
spec.skip_validation:` gate
258         # against a refactor that appends the flag unconditionally (which would silently
bypass the direct
259         # `worker infer` cloud path's validation ? the very thing #skip-validation is scoped
away from).
260         assert "--skip-validation" not in argv
261         # the container-inline overrides come last (win) and the user override is carried
through
262         joined = " ".join(argv)
263         assert "deployment.compute_target=local_gpu" in joined
264         assert "indexing.detector_impl=native" in joined
265         assert "sport=tennis" in joined
266         # user override precedes the inline-forcing overrides in --set order
267         assert joined.index("sport=tennis") < joined.index(
268             "deployment.compute_target=local_gpu"
269         )
270
271
272     def test_dispatch_argv_carries_skip_validation_when_set(tmp_path):
273         """The builder must emit --skip-validation when the spec opts out ? the control-
plane sets
274         skip_validation=True on every dispatch (it is the authoritative validation gate),
and the worker
275         honours the flag to skip its redundant re-validation. Positive builder-level guard,
independent of
276         the control-plane path (which hardcodes True)."""
277         client = _FakeRunClient()
278         backend = CloudRunCompute(
279             run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
280         )
281         backend.run_infer(
282             ComputeJobSpec(
283                 video_uri="gs://b/v.mp4",
284                 out_uri=str(tmp_path / "bk"),
285                 skip_validation=True,
286             )
287         )
288         assert "--skip-validation" in client.calls[0]["argv"]
289
290
291     def test_unique_token_per_call_when_not_injected(tmp_path):
292         """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
293         client = _FakeRunClient()
294         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
295         backend.run_infer(
296             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
297         )
298         backend.run_infer(
299             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
300         )
301         done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
302         done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
303         assert done1 != done2
304
305
306     def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
307         """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a None dst raises) ?
_read_json must
308         pass one, else every real cloud marker/report read silently returns {} (local hid
this via its
309         identity passthrough). Mimic a cloud backend that rejects dst=None."""
310         from backend.compute import cloud_run as cr
311

```



```

312     captured = {}
313
314     def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
315         captured["dst"] = dst
316         if dst is None:
317             raise ValueError(
318                 "cloud backend requires a dst path"
319             ) # mirror GCSStorage/S3Storage
320         with open(dst, "w", encoding="utf-8") as f:
321             f.write('{"video_id": "v", "returncode": 0}')
322         return dst
323
324     monkeypatch.setattr(cr, "fetch", fake_fetch)
325     backend = cr.CloudRunCompute(run_client=_FakeRunClient(), job_name="j", region="r")
326     data = backend._read_json("gs://b/x.json")
327     assert (
328         captured["dst"] is not None
329     ) # the fix: a real dst is always passed (no silent {})
330     assert data == {"video_id": "v", "returncode": 0}
331     assert not os.path.exists(os.path.dirname(captured["dst"]))
332
333
334     def test_cloud_run_polls_until_marker_appears(monkeypatch):
335         """The bucket-poll WAIT path (D7 scale-to-zero cold start): the marker is ABSENT for
the first
336         polls then appears ? poll_until must LOOP, not resolve on poll 0. The synchronous
fake elsewhere
337         never exercises this; here exists() is False twice then True."""
338         from backend.compute import cloud_run as cr
339
340         exists_calls = {"n": 0}
341
342         class _FakeStore:
343             def exists(self, ref):
344                 exists_calls["n"] += 1
345                 return exists_calls["n"] >= 3 # absent for polls 1-2, present on poll 3
346
347         monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
348
349         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
350             if dst is None:
351                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-poll-"), "m.json")
352                 with open(dst, "w", encoding="utf-8") as f:
353                     f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
354                 return dst
355
356         monkeypatch.setattr(cr, "fetch", fake_fetch)
357         backend = cr.CloudRunCompute(
358             run_client=_NoopRunClient(),
359             job_name="j",
360             region="r",
361             token="t",
362             policy=ExecutionPolicy(
363                 poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0
364             ),
365         )
366         result = backend.run_infer(
367             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
368         )
369         assert result.video_id == "vlate"
370         assert (
371             exists_calls["n"] >= 3
372         ) # >=1 absent poll occurred before the marker appeared (loop ran)
373
374

```

```

375     # ----- poll timeout: rescue-or-cancel (R1-21 GPU
cost leak)
376
377     # max_wait_sec=0.0 ? poll_until gives up after its FIRST absent check (no real waiting).
378     _TIMEOUT_NOW = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0,
max_wait_sec=0.0)
379
380
381     def test_poll_timeout_cancels_execution_and_raises(tmp_path):
382         """Poll deadline + no marker ? the still-running execution is cancelled (with the
EXACT name
383         run_job returned) and PolicyTimeout surfaces carrying that name for the operator."""
384         client = _NeverDoneClient()
385         backend = CloudRunCompute(
386             run_client=client,
387             job_name="rally-worker",
388             region="asia-south1",
389             policy=_TIMEOUT_NOW,
390             token="tok",
391         )
392         with pytest.raises(PolicyTimeout) as exc:
393             backend.run_infer(
394                 ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
395             )
396         assert client.cancelled == [_NeverDoneClient.EXEC_NAME] # cancel got the right name
397         msg = str(exc.value)
398         assert _NeverDoneClient.EXEC_NAME in msg # operator can find the execution in GCP
399         assert "cancel requested" in msg
400         assert "max_wait_sec" in msg # the original timeout text is preserved, not masked
401
402
403     def test_poll_timeout_cancel_failure_still_raises_policy_timeout(tmp_path):
404         """Best-effort means BEST-EFFORT: a cancel API failure must never mask the original
timeout ?
405         the caller (main.py's dispatch handler) still sees PolicyTimeout, with the failure
noted."""
406         client = _NeverDoneClient(cancel_error=RuntimeError("cancel API unreachable"))
407         backend = CloudRunCompute(
408             run_client=client,
409             job_name="rally-worker",
410             region="asia-south1",
411             policy=_TIMEOUT_NOW,
412             token="tok",
413         )
414         with pytest.raises(PolicyTimeout) as exc:
415             backend.run_infer(
416                 ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
417             )
418         assert client.cancelled == [_NeverDoneClient.EXEC_NAME] # the attempt WAS made
419         msg = str(exc.value)
420         assert _NeverDoneClient.EXEC_NAME in msg
421         assert "cancel API unreachable" in msg # the cancel failure is surfaced, not
swallowed
422         assert "max_wait_sec" in msg # ...alongside the original timeout text
423
424
425     def test_poll_timeout_late_marker_rescues_result(monkeypatch):
426         """A slow-but-healthy worker that finishes JUST past the poll deadline
(max_wait_sec=1800 can
427         undercut the job's --task-timeout=3600) is rescued by the ONE final post-timeout
marker check:
428         the completed GPU result is ingested, nothing is cancelled, no timeout surfaces."""
429         from backend.compute import cloud_run as cr
430
431         exists_calls = {"n": 0}

```

```

432
433     class _FakeStore:
434         def exists(self, ref):
435             exists_calls["n"] += 1
436             return exists_calls["n"] >= 2 # absent for the poll, present on the final
check
437
438     monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
439
440     def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
441         if dst is None:
442             dst = os.path.join(tempfile.mkdtemp(prefix="cr-late-"), "m.json")
443             with open(dst, "w", encoding="utf-8") as f:
444                 f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
445             return dst
446
447     monkeypatch.setattr(cr, "fetch", fake_fetch)
448     client = _NoopRunClient()
449     backend = cr.CloudRunCompute(
450         run_client=client, job_name="j", region="r", token="t", policy=_TIMEOUT_NOW
451     )
452     result = backend.run_infer(
453         ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
454     )
455     assert result.returncode == 0
456     assert result.video_id == "vlate" # the late result is ingested, not wasted
457     assert client.cancelled == [] # a finished execution is never cancelled
458     assert exists_calls["n"] == 2 # exactly ONE extra post-timeout check
459
460
461     # ----- execution-status fast negative path (worker crashed
pre-marker)
462
463     # poll_every_n_sec=0.0 ? the poll spins as fast as possible; max_wait_sec=2.0 BOUNDS any
regression
464     # (a fast path that silently fell back to marker-only polling fails in ~2s here, never a
65-min hang)
465     # while the working fast path aborts on poll 0 ? instantly.
466     _FAILFAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=2.0)
467
468
469     def test_run_infer_fails_fast_when_execution_terminal_without_marker(tmp_path):
470         """The core fix: a worker execution that TERMINATED (Failed) without ever writing a
marker is
471         caught by the Executions-API probe ? RemoteExecutionFailed FAST (poll 0), not a
~65-min opaque
472         hang. The diagnostic names the execution, the terminal state, and the log-inspection
command."""
473         client = _CrashedNoMarkerClient(state="Failed")
474         backend = CloudRunCompute(
475             run_client=client,
476             job_name="rally-worker",
477             region="asia-south1",
478             policy=_FAILFAST,
479             token="tok",
480         )
481         with pytest.raises(RemoteExecutionFailed) as exc:
482             backend.run_infer(
483                 ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
484             )
485         msg = str(exc.value)
486         assert _CrashedNoMarkerClient.EXEC_NAME in msg # operator can find the execution
487         assert "Failed" in msg # the terminal state is surfaced
488         assert "gcloud run jobs executions describe" in msg # actionable log pointer
489         assert client.status_polls == 1 # aborted on the FIRST terminal observation (fast,

```

```

not spun)
490         assert client.cancelled == [] # a finished execution is never cancelled (nothing to
cancel)
491
492
493     def test_dispatch_remote_maps_terminal_without_marker_to_rc5(tmp_path):
494         """The dispatch handler maps RemoteExecutionFailed to a distinct, clearly-labelled
rc 5 ? NOT
495         the rc 4 poll-timeout path (a still-maybe-running execution) and NOT a raw traceback
? so
496         'worker crashed before the marker' is operationally distinguishable and surfaces
immediately."""
497         import argparse
498
499         from backend.worker import __main__ as wmain
500
501         client = _CrashedNoMarkerClient(state="Failed")
502         backend = CloudRunCompute(
503             run_client=client,
504             job_name="rally-worker",
505             region="asia-south1",
506             policy=_FAILFAST,
507             token="tok",
508         )
509         args = argparse.Namespace(
510             video_uri="gs://b/v.mp4",
511             out_uri=str(tmp_path / "bucket"),
512             segmenter="wasb_hybrid",
513             set=None,
514         )
515         rc = wmain._dispatch_remote(backend, args)
516         assert rc == 5 # the crashed-pre-marker fast-fail code...
517         assert rc != 4 # ...distinct from the poll-budget timeout
518         assert client.status_polls == 1 # reached the fast path (not the timeout branch)
519
520
521     def test_succeeded_state_without_marker_also_fails_fast(tmp_path):
522         """Generalization: ANY terminal state with no marker is a guaranteed-futile wait. A
'Succeeded'
523         execution that nonetheless wrote no marker (the marker upload itself failed) also
fails fast,
524         with the state surfaced ? the whole 'execution over, no marker' class collapses, not
just Failed."""
525         client = _CrashedNoMarkerClient(state="Succeeded")
526         backend = CloudRunCompute(
527             run_client=client,
528             job_name="rally-worker",
529             region="asia-south1",
530             policy=_FAILFAST,
531             token="tok",
532         )
533         with pytest.raises(RemoteExecutionFailed) as exc:
534             backend.run_infer(
535                 ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
536             )
537         assert "Succeeded" in str(exc.value)
538
539
540     def test_present_marker_wins_over_terminal_status(tmp_path):
541         """The done-marker stays AUTHORITATIVE: a worker that wrote a marker ? even a
FAILURE marker
542         (rc 2) ? resolves through the normal path with its REAL returncode, never a
synthesized
543         RemoteExecutionFailed, even when the execution status also reads terminal. Status is
a negative

```

```

544     path only; it must never override a real marker (which is checked first)."""
545
546     class _MarkerAndFailedStatus(_FakeRunClient):
547         def execution_terminal_state(self, execution): # pragma: no cover - never
reached
548             return "Failed" # the marker resolves poll 0, so this is never consulted
549
550     client = _MarkerAndFailedStatus(video_id="abc", returncode=2)
551     backend = CloudRunCompute(
552         run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
553     )
554     result = backend.run_infer(
555         ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
556     )
557     assert result.returncode == 2 # the worker's REAL rc, not RemoteExecutionFailed
558     assert result.video_id == "abc"
559
560
561     def test_status_probe_error_is_swallowed_and_falls_back_to_marker_polling(tmp_path):
562         """Best-effort probe: a raising Executions API must NEVER fail a healthy run. The
wait degrades
563         to today's marker-only polling ? PolicyTimeout (the existing timeout path), NOT
564         RemoteExecutionFailed ? a flaky status API cannot manufacture a failure."""
565
566         class _RaisingStatusClient(_NeverDoneClient):
567             def execution_terminal_state(self, execution):
568                 raise RuntimeError("executions API unreachable")
569
570         client = _RaisingStatusClient()
571         backend = CloudRunCompute(
572             run_client=client,
573             job_name="rally-worker",
574             region="asia-south1",
575             policy=_TIMEOUT_NOW,
576             token="tok",
577         )
578         with pytest.raises(PolicyTimeout): # swallowed probe error ? the unchanged timeout
path
579             backend.run_infer(
580                 ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
581             )
582         assert client.cancelled == [_NeverDoneClient.EXEC_NAME] # still cancels on the real
timeout
583
584
585     def test_default_execution_terminal_state_is_none():
586         """The ABC default is None (additive / default-OFF): a client that doesn't override
status
587         polling degrades transparently to marker-only behaviour, so every pre-existing fake
is
588         unaffected and the seam never changes a healthy run's outcome."""
589         assert _NoopRunClient().execution_terminal_state("whatever") is None
590         assert (
591             _NeverDoneClient().execution_terminal_state(_NeverDoneClient.EXEC_NAME) is None
592         )
593
594
595     def test_real_client_terminal_state_none_for_unresolvable_name():
596         """The real client's status probe is best-effort: a non-resolvable execution name
returns None
597         (can't probe ? keep marker-polling), NOT a raise ? exercisable in CI (the SDK is
absent)."""
598         from backend.compute.cloud_run import GoogleCloudRunJobClient
599
600         client = GoogleCloudRunJobClient()

```

```

601     for bogus in ("", "execution", "exec-123"):
602         assert client.execution_terminal_state(bogus) is None
603
604
605     def test_terminal_status_with_marker_written_in_race_resolves_via_marker(monkeypatch):
606         """Concurrency race: the execution reads terminal exactly as the worker finishes
writing its
607         marker. _poll_check re-checks the marker ONCE after a terminal status ? if it landed
in between,
608         the completed result is ingested (the marker wins), NOT a spurious
RemoteExecutionFailed."""
609         from backend.compute import cloud_run as cr
610
611         exists_calls = {"n": 0}
612
613         class _FakeStore:
614             def exists(self, ref):
615                 exists_calls["n"] += 1
616                 # absent on the pre-status read, present on the ONE post-status race re-
check
617                 return exists_calls["n"] >= 2
618
619         monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
620
621         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
622             if dst is None:
623                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-race-"), "m.json")
624             with open(dst, "w", encoding="utf-8") as f:
625                 f.write('{"video_id": "vrace", "returncode": 0, "segment_count": 1}')
626             return dst
627
628         monkeypatch.setattr(cr, "fetch", fake_fetch)
629         client = _CrashedNoMarkerClient(state="Failed") # status says terminal-Failed...
630         backend = cr.CloudRunCompute(
631             run_client=client, job_name="j", region="r", token="t", policy=_FAILFAST
632         )
633         result = backend.run_infer(
634             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
635         )
636         assert result.video_id == "vrace" # ...but the marker that raced in WINS
637         assert exists_calls["n"] == 2 # exactly the pre-status read + the ONE race re-check
638         assert client.status_polls == 1 # status was consulted once, then the marker
superseded it
639
640
641     def test_terminal_execution_state_none_for_empty_execution():
642         """An empty/unknown execution name (run_job could not resolve one) can't be probed ?
None
643         (degrade to marker-only polling), and the client's status API is never even
called."""
644
645         class _Boom(_NoopRunClient):
646             def execution_terminal_state(self, execution): # pragma: no cover - must never
be called
647                 raise AssertionError("must not probe an empty execution name")
648
649         backend = CloudRunCompute(run_client=_Boom(), job_name="j", region="r")
650         assert backend._terminal_execution_state("") is None
651
652
653     # ----- worker wiring: dispatch branch +
done-marker
654
655
656     def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):

```

```

657     """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
658     from backend.compute.base import ComputeResult
659     from backend.worker import __main__ as wmain
660
661     seen = {}
662
663     class _FakeBackend:
664         def run_infer(self, spec):
665             seen["spec"] = spec
666             return ComputeResult(
667                 returncode=0, video_id="vidX", outputs_uri="gs://b/outputs/vidX/"
668             )
669
670     monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
671     # cloud-serving startup checks require the AI-handoff key (AI_HANDOFF_KEY_MISSING is
fatal).
672     monkeypatch.setenv("GEMINI_API_KEY", "test-key")
673     cfg = tmp_path / "c.json"
674     cfg.write_text(
675         '{"deployment": {"profile": "cloud-serving"}, "storage": {"backend": "gcs"}}'
676     )
677     rc = wmain.cmd_infer(
678         wmain.build_parser().parse_args(
679             [
680                 "infer",
681                 "--video-uri",
682                 "gs://b/videos/clip.mp4",
683                 "--out-uri",
684                 "gs://b",
685                 "--config",
686                 str(cfg),
687                 "--segmenter",
688                 "wasb_hybrid",
689             ]
690         )
691     )
692     assert rc == 0
693     assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
694     assert seen["spec"].segmenter == "wasb_hybrid"
695
696
697     def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
698         """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
699         remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
700         from backend.worker import __main__ as wmain
701
702         class _OK:
703             passed = True
704             validator_name = "fake"
705             message = "ok"
706
707             def model_dump(self):
708                 return {"validator_name": "fake", "passed": True, "message": "ok"}
709
710         monkeypatch.setattr(
711             wmain.validator_registry,
712             "run_all_with_context",
713             lambda path, cfg: ([_OK()], {}),
714         )
715         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
716         monkeypatch.setattr(
717             "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",

```

```

718         lambda p: 30.0,
719     )
720
721     cfg = tmp_path / "c.json"
722     cfg.write_text(
723         json.dumps(
724             {
725                 "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
726                 "output": {"output_dir": str(tmp_path / "o")},
727             }
728         )
729     )
730     video = tmp_path / "in.mp4"
731     video.write_bytes(b"FAKEVIDEO")
732     out = tmp_path / "bucket"
733     done = str(out / "_dispatch" / "tok.done")
734
735     rc = wmain.cmd_infer(
736         wmain.build_parser().parse_args(
737             [
738                 "infer",
739                 "--video-uri",
740                 str(video),
741                 "--out-uri",
742                 str(out),
743                 "--config",
744                 str(cfg),
745                 "--scratch",
746                 str(tmp_path / "s"),
747                 "--segmenter",
748                 "wasb_hybrid",
749                 "--done-uri",
750                 done,
751             ]
752         )
753     )
754     assert rc == 0
755     marker = json.loads(open(done, encoding="utf-8").read())
756     assert marker["returncode"] == 0 and marker["status"] == "success"
757     assert marker["video_id"] and marker["segment_count"] >= 1
758     # CP-rebuild P3 (#485): the report records the SOURCE the detection ran on, so a
759     # restart-wiped control plane re-hydrating from outputs/<md5>/ can restore the video
760     # row's truthful path (not just the segments).
761     report_path = out / "outputs" / marker["video_id"] / "report.json"
762     report = json.loads(report_path.read_text(encoding="utf-8"))
763     assert report["video_uri"] == str(video)
764
765
766     def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
767         """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
768         behaviour)."""
769         from backend.worker import __main__ as wmain
770
771         class _OK:
772             passed = True
773             validator_name = "fake"
774             message = "ok"
775
776             def model_dump(self):
777                 return {"validator_name": "fake", "passed": True, "message": "ok"}
778
779         monkeypatch.setattr(
780             wmain.validator_registry,
781             "run_all_with_context",
782             lambda path, cfg: ([_OK()], {}),

```



```

782     )
783     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
784     monkeypatch.setattr(
785         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
786         lambda p: 30.0,
787     )
788     cfg = tmp_path / "c.json"
789     cfg.write_text(
790         json.dumps(
791             {
792                 "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
793                 "output": {"output_dir": str(tmp_path / "o")},
794             }
795         )
796     )
797     video = tmp_path / "in.mp4"
798     video.write_bytes(b"FAKEVIDEO")
799     out = tmp_path / "bucket"
800     rc = wmain.cmd_infer(
801         wmain.build_parser().parse_args(
802             [
803                 "infer",
804                 "--video-uri",
805                 str(video),
806                 "--out-uri",
807                 str(out),
808                 "--config",
809                 str(cfg),
810                 "--scratch",
811                 str(tmp_path / "s"),
812                 "--segmenter",
813                 "wasb_hybrid",
814             ]
815         )
816     )
817     assert rc == 0
818     assert not (out / "_dispatch").exists() # no marker dir when --done-uri is omitted
819
820
821     def test_worker_writes_failure_marker_on_validation_fail(tmp_path, monkeypatch):
822         """The container side: a worker-side FAILURE (validation rc=2) with --done-uri
823         writes a marker
824         carrying the REAL rc, so a remote dispatcher's bucket-poll terminates FAST (not a
825         30-min hang)."""
826         from backend.worker import __main__ as wmain
827
828         class _BAD:
829             passed = False
830             validator_name = "blur"
831             message = "blurry"
832
833             def model_dump(self):
834                 return {"validator_name": "blur", "passed": False, "message": "blurry"}
835
836         monkeypatch.setattr(
837             wmain.validator_registry,
838             "run_all_with_context",
839             lambda path, cfg: ([_BAD()], {}),
840         )
841         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
842         cfg = tmp_path / "c.json"
843         cfg.write_text(
844             json.dumps(
845                 {
846                     "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},

```

```

845         "output": {"output_dir": str(tmp_path / "o")},
846     }
847 )
848 )
849 video = tmp_path / "in.mp4"
850 video.write_bytes(b"FAKEVIDEO")
851 out = tmp_path / "bucket"
852 done = str(out / "_dispatch" / "tok.done")
853
854 rc = wmain.cmd_infer(
855     wmain.build_parser().parse_args(
856         [
857             "infer",
858             "--video-uri",
859             str(video),
860             "--out-uri",
861             str(out),
862             "--config",
863             str(cfg),
864             "--scratch",
865             str(tmp_path / "s"),
866             "--segmenter",
867             "wasb_hybrid",
868             "--done-uri",
869             done,
870         ]
871     )
872 )
873 assert rc == 2 # validation failure
874 marker = json.loads(open(done, encoding="utf-8").read())
875 assert marker["returncode"] == 2 and marker["status"] == "failed"
876 assert marker[
877     "video_id"
878 ] # carries the id so the dispatcher's poll terminates (rc != 0 ? failed)
879
880
881 def test_worker_failure_writes_no_marker_without_done_uri(tmp_path, monkeypatch):
882     """DEFAULT-OFF: a worker failure with NO --done-uri writes no marker (the unchanged
883     local run)."""
884     from backend.worker import __main__ as wmain
885
886     class _BAD:
887         passed = False
888         validator_name = "blur"
889         message = "blurry"
890
891         def model_dump(self):
892             return {"validator_name": "blur", "passed": False, "message": "blurry"}
893
894     monkeypatch.setattr(
895         wmain.validator_registry,
896         "run_all_with_context",
897         lambda path, cfg: ([_BAD()], {}),
898     )
899     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
900     cfg = tmp_path / "c.json"
901     cfg.write_text(
902         json.dumps(
903             {
904                 "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
905                 "output": {"output_dir": str(tmp_path / "o")},
906             }
907         )
908     )
909     video = tmp_path / "in.mp4"

```

```

909     video.write_bytes(b"FAKEVIDEO")
910     out = tmp_path / "bucket"
911     rc = wmain.cmd_infer(
912         wmain.build_parser().parse_args(
913             [
914                 "infer",
915                 "--video-uri",
916                 str(video),
917                 "--out-uri",
918                 str(out),
919                 "--config",
920                 str(cfg),
921                 "--scratch",
922                 str(tmp_path / "s"),
923                 "--segmenter",
924                 "wasb_hybrid",
925             ]
926         )
927     )
928     assert rc == 2
929     assert not (out / "_dispatch").exists()
930
931
932     # ----- Dockerfile structural guard
933     (no build)
934
935     def test_dockerfile_has_required_directives():
936         """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
937         structurally:
938         the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
939         worker entrypoint."""
940         from pathlib import Path
941
942         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
943         assert "FROM nvidia/cuda" in text # GPU base (CUDA)
944         assert "ffmpeg" in text # stitch/decode
945         assert "fonts-dejavu" in text # the watermark filter font
946         # The #506 modernized WASB stack (L4-validated): torch 2.6 cu124 + PyNvVideoCodec,
947         and
948         # the NVDEC/NVENC driver userspace libs PyNvVideoCodec dlopens (compute-only GPU
949         hosts
950         # inject neither) ? a silent drop of any of these bricks
951         decode_backend=nvdec_resident.
952         assert "torch==2.6.0" in text
953         assert "download.pytorch.org/whl/cu124" in text
954         assert "PyNvVideoCodec" in text
955         assert "libnvcuvid.so.1" in text and "libnvidia-encode.so.1" in text
956         assert "torch==1.11.0+cu113" not in text # the old stack is gone, not co-installed
957         assert "WASB-SBDT" in text # the shuttle detector repo
958         assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
959         assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
960         # The cloud worker reads the gs:// input + writes the gs:// outputs/done-marker
961         through
962         # backend.storage (storage-gcs); and when this image is reused as the deployed
963         control plane it
964         # dispatches the L4 Job via google.cloud.run_v2 (cloud-run) and verifies Cf-Access
965         JWTs when
966         # edge-auth is on (auth ? PyJWT). A bare `pip install .` boots but dies on the first
967         gs:// fetch
968         # (ModuleNotFoundError: google.cloud), on dispatch (google-cloud-run missing), or on
969         the first
970         # edge-authed request (jwt missing) ? real Cloud Run failures the mock-client tests
971         can't see.
972         # Lock the extras in structurally.

```

```

962         assert ".[storage-gcs,cloud-run,auth]" in text
963
964
965     def test_dockerfile_uses_micromamba_not_anaconda_tos():
966         """P3a: the WASB env is built with micromamba + conda-forge (BSD, no ToS), NOT
Miniconda +
967         `conda tos accept` (the 2000-employee Anaconda Terms-of-Service trap,
PACKAGING_PLAN.md §3).
968         Locked structurally so a revert to the ToS path fails CI; the env path (WASB_PYTHON)
is unchanged."""
969         from pathlib import Path
970
971         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
972         assert "micromamba" in text # the conda-forge env builder
973         assert "conda-forge" in text # the BSD, ToS-free channel
974         assert (
975             "MAMBA_ROOT_PREFIX=/opt/conda" in text
976         ) # keeps the env at /opt/conda/envs/wasb
977         assert "Miniconda3" not in text # the Anaconda installer is gone
978         assert "conda tos accept" not in text # no ToS acceptance needed (conda-forge)
979
980
981     def test_remote_poll_budget_comes_from_config_unless_policy_injected():
982         """CP-rebuild §4.3: deployment.remote_poll_max_wait_sec drives the dispatch poll
budget
983         (default 3900s > the Job's --task-timeout 3600s ? the old 1800s default could
abandon a
984         slow-but-healthy paid GPU run), and an explicitly injected policy always wins."""
985         from backend.infrastructure.execution_policy import DEFAULT_POLICY
986
987         cfg = AppConfig()
988         cfg.deployment.compute_target = "cloud_run"
989
990         be = get_compute_backend(cfg, run_client=_FakeRunClient())
991         assert be is not None
992         assert be._policy.max_wait_sec == 3900.0
993         # Only the budget is derived ? the other resilience knobs stay at their defaults.
994         assert be._policy.poll_every_n_sec == DEFAULT_POLICY.poll_every_n_sec
995         assert be._policy.op_timeout_sec == DEFAULT_POLICY.op_timeout_sec
996
997         cfg.deployment.remote_poll_max_wait_sec = 42.0
998         assert get_compute_backend(cfg, run_client=_FakeRunClient())._policy.max_wait_sec ==
42.0
999
1000         # An injected policy is never overridden by config.
1001         be = get_compute_backend(cfg, run_client=_FakeRunClient(), policy=_FAST)
1002         assert be._policy.max_wait_sec == _FAST.max_wait_sec
1003
1004
1005     def test_run_infer_forwards_on_wait_into_the_bucket_poll(tmp_path, monkeypatch):
1006         """The #482 heartbeat plumbing: the on_wait callback handed to run_infer must be the
exact on_tick the bucket-poll drives (cloud_run threads it through poll_until)."""
1007         from backend.compute import cloud_run as cr
1008
1009
1010         seen = {}
1011
1012         def fake_poll(check, policy, *, label="op", logger=None, on_tick=None, **kw):
1013             seen["on_tick"] = on_tick
1014             return {"video_id": "vidCloud", "returncode": 0}
1015
1016         monkeypatch.setattr(cr, "poll_until", fake_poll)
1017         client = _FakeRunClient() # still writes report.json so the post-marker read
succeeds
1018         backend = CloudRunCompute(
1019             run_client=client,

```

```

1020         job_name="rally-worker",
1021         region="asia-southeast1",
1022         storage_config={},
1023         policy=_FAST,
1024         token="tok",
1025     )
1026
1027     def heartbeat(_elapsed):
1028         pass
1029
1030     result = backend.run_infer(
1031         ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk")),
1032         on_wait=heartbeat,
1033     )
1034     assert seen["on_tick"] is heartbeat
1035     assert result.video_id == "vidCloud"
1036

```

7. user (2026-07-07T15:48:25.011Z)

```

1     """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11        job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12        worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13        the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15        Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16        worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17        (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
25    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26    owner-verified on the M7 real run.
27    """

```

```

28
29     from __future__ import annotations
30
31     import json
32     import logging
33     import os
34     import tempfile
35     import uuid
36     from abc import ABC, abstractmethod
37     from typing import Any, Callable, List, Optional
38
39     from backend.compute.base import (
40         ComputeBackend,
41         ComputeJobSpec,
42         ComputeResult,
43         RemoteExecutionFailed,
44     )
45     from backend.infrastructure.execution_policy import (
46         DEFAULT_POLICY,
47         ExecutionPolicy,
48         PolicyTimeout,
49         poll_until,
50     )
51     from backend.storage import fetch, get_storage, parse_uri
52
53     LOG = logging.getLogger("compute.cloud_run")
54
55     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
56     # branch.
57     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
58     # runner.)
59     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
60         "deployment.compute_target=local_gpu",
61         "indexing.detector_impl=native",
62     )
63
64     class CloudRunJobClient(ABC):
65         """Triggers a Cloud Run **Job** execution with per-execution container arg
66         overrides.
67
68         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
69         implementation
70         overrides the job's container ``args`` for this execution and starts it."""
71
72         @abstractmethod
73         def run_job(
74             self,
75             job_name: str,
76             argv: List[str],
77             *,
78             region: str,
79             project: Optional[str] = None,
80         ) -> str:
81             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
82             return an
83             execution id/name (for logs). Does NOT block on completion (the caller bucket-
84             polls)."""
85
86         @abstractmethod
87         def cancel_execution(self, execution: str) -> None:
88             """Cancel a running execution by the name ``run_job`` returned. Called when the
89             bucket-poll
90             times out so the abandoned execution doesn't keep billing the GPU until
91             ``--task-timeout``.

```

```

85         May raise ? the caller treats every failure as best-effort (the original poll
timeout is
86         NEVER masked by a cancel error)."""
87
88     def execution_terminal_state(self, execution: str) -> Optional[str]:
89         """Return the execution's TERMINAL state as a short label (`"Failed"` /
`"Cancelled"` /
90         `"Succeeded"`) if it has finished, else `None` (still pending/running, or
the state
91         can't be determined).
92
93         ADVISORY, not authoritative: it feeds `run_infer`'s fast negative path so a
worker that
94         terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95         of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97         NON-abstract with a default of `None` (additive / default-OFF): a client that
can't ? or
98         chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99         polling. Implementations MAY raise on a transient API error ? `run_infer`
treats a raise
100        as `None` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101        return None
102
103
104    class GoogleCloudRunJobClient(CloudRunJobClient):
105        """Real client ? lazy-imports `google-cloud-run`. Owner-verified on the M7 real
run; CI uses
106        a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108        def run_job(
109            self,
110            job_name: str,
111            argv: List[str],
112            *,
113            region: str,
114            project: Optional[str] = None,
115        ) -> str:
116            # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
117            # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
118            # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
119            if not project:
120                raise RuntimeError(
121                    "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
122                    "to resolve the job path; set it on the control plane (see
get_compute_backend)."
123                )
124            try:
125                from google.cloud import run_v2 # type: ignore[attr-defined]
126            except Exception as exc: # pragma: no cover - real SDK not present in CI
127                raise RuntimeError(
128                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
129                    "control plane (it is intentionally NOT a CI/test dependency)."
130                ) from exc
131            client = (

```

```

132         run_v2.JobsClient()
133     ) # pragma: no cover - exercised only on the real M7 run
134     name = client.job_path(project, region, job_name)
135     overrides = run_v2.RunJobRequest.Overrides(
136         container_overrides=[
137             run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
138         ]
139     )
140     op = client.run_job(
141         request=run_v2.RunJobRequest(name=name, overrides=overrides)
142     )
143     return (
144         getattr(op, "metadata", None)
145         and getattr(op.metadata, "name", "")
146         or "execution"
147     )
148
149     def cancel_execution(self, execution: str) -> None:
150         # run_job falls back to the literal string "execution" when the operation
151         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
152         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
153         # google-cloud-run is absent).
154         if "/executions/" not in (execution or ""):
155             raise RuntimeError(
156                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
157                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
158                 "one from the operation metadata, so there is nothing to cancel by
159                 )
160             try:
161                 from google.cloud import run_v2 # type: ignore[attr-defined]
162             except Exception as exc: # pragma: no cover - real SDK not present in CI
163                 raise RuntimeError(
164                     "google-cloud-run is required to cancel a Cloud Run execution; install
165                     "control plane (it is intentionally NOT a CI/test dependency).")
166                 ) from exc
167             client = (
168                 run_v2.ExecutionsClient()
169             ) # pragma: no cover - exercised only on a real run
170             client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
171
172     def execution_terminal_state(self, execution: str) -> Optional[str]:
173         # Advisory status probe (see the ABC docstring): describe the execution and,
174         # is terminal (`completion_time` set), classify it. A non-resolvable name /
175         # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this
176         # raises on bad input, because the probe is best-effort and must not fail a
177         # healthy run.
178         if "/executions/" not in (execution or ""):
179             return None
180         try:
181             from google.cloud import run_v2 # type: ignore[attr-defined]
182         except Exception: # pragma: no cover - real SDK not present in CI
183             return None
184         client = (
185             run_v2.ExecutionsClient()
186         ) # pragma: no cover - exercised only on a real run

```



```

186         exe = client.get_execution( # pragma: no cover - real run only
187             request=run_v2.GetExecutionRequest(name=execution)
188         )
189         # completion_time is a proto Timestamp: unset ? seconds==0 ? still
running/pending.
190         if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
191             return None # pragma: no cover - real run only
192         # Terminal: prefer the explicit per-task counts (a cancel shows as
cancelled_count>0).
193         if getattr(exe, "cancelled_count", 0): # pragma: no cover - real run only
194             return "Cancelled"
195         if getattr(exe, "failed_count", 0): # pragma: no cover - real run only
196             return "Failed"
197         if getattr(exe, "succeeded_count", 0): # pragma: no cover - real run only
198             return "Succeeded"
199         return "Failed" # pragma: no cover - terminal but no positive success ? treat
as failed
200
201
202     class CloudRunCompute(ComputeBackend):
203         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
204
205         def __init__(
206             self,
207             *,
208             run_client: CloudRunJobClient,
209             job_name: str,
210             region: str,
211             project: Optional[str] = None,
212             storage_config: Optional[dict] = None,
213             policy: ExecutionPolicy = DEFAULT_POLICY,
214             token: Optional[str] = None,
215             container_overrides: Optional[tuple[str, ...]] = None,
216         ) -> None:
217             self._client = run_client
218             self._job_name = job_name
219             self._region = region
220             self._project = project
221             self._storage_config = storage_config or {}
222             self._policy = policy
223             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
224             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
225             self._token = token
226             self._container_overrides = (
227                 _DEFAULT_CONTAINER_OVERRIDES
228                 if container_overrides is None
229                 else tuple(container_overrides)
230             )
231
232         def run_infer(
233             self,
234             spec: ComputeJobSpec,
235             *,
236             on_wait: "Callable[[float], None] | None" = None,
237         ) -> ComputeResult:
238             out_root = spec.out_uri.rstrip("/")
239             token = self._token or uuid.uuid4().hex
240             done_uri = f"{out_root}/_dispatch/{token}.done"
241             argv: List[str] = [
242                 "infer",
243                 "--video-uri",
244                 spec.video_uri,
245                 "--out-uri",

```

```

246         spec.out_uri,
247         "--done-uri",
248         done_uri,
249     ]
250     if spec.segmenter:
251         argv += ["--segmenter", spec.segmenter]
252     if spec.skip_validation:
253         # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
254         # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
255         argv.append("--skip-validation")
256     for kv in (*spec.config_overrides, *self._container_overrides):
257         argv += ["--set", kv]
258
259     execution = self._client.run_job(
260         self._job_name, argv, region=self._region, project=self._project
261     )
262     LOG.info(
263         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
264         self._job_name,
265         execution,
266         done_uri,
267     )
268
269     try:
270         marker = poll_until(
271             lambda: self._poll_check(done_uri, str(execution or "")),
272             self._policy,
273             label="cloud-run-infer",
274             logger=LOG,
275             # Live heartbeat (#482): each still-waiting tick reaches the caller so
276             # job.progress moves during the whole GPU run instead of freezing.
277             on_tick=on_wait,
278         )
279     except PolicyTimeout as timeout_exc:
280         # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
281         # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
282         # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
283         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
284         video_id = str(marker.get("video_id", ""))
285         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
286         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
287         rc = int(marker.get("returncode", 1))
288         if not video_id:
289             LOG.warning(
290                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
291                 "video_id) ? treating as failure",
292                 self._job_name,
293             )
294             rc = rc or 1
295         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
296         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
297         LOG.info(
298             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
299         )
300     return ComputeResult(
301         returncode=rc,
302         outputs_uri=outputs_uri,
303         video_id=video_id,

```

```

304         report=report,
305         execution=str(execution or ""),
306     )
307
308     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
309         """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
310
311         The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
312         wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
313         FAILURE* marker resolves through the normal path with its real returncode. Only
when the
314         marker is absent do we consult the execution status: a worker that TERMINATED
without ever
315         writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
316         otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
317         opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
318         failure.
319
320         Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
321         ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
322         will ever appear)."""
323         marker = self._read_marker(done_uri)
324         if marker is not None:
325             return marker # success signal ? authoritative even if status later flips
326         state = self._terminal_execution_state(execution)
327         if state is None:
328             return None # still pending/running, or status unknown ? keep bucket-
polling
329         # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
330         # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
331         # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
332         # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
333         marker = self._read_marker(done_uri)
334         if marker is not None:
335             return marker
336         raise RemoteExecutionFailed(
337             f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
338             f"terminal state {state!r} with NO completion marker ? the worker exited
before "
339             "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
340             "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
341             "inspect the Cloud Run Job execution log to diagnose: "
342             f"`gcloud run jobs executions describe {execution or '<execution>'} "
343             f"--region {self._region}`."
344         )
345
346     def _terminal_execution_state(self, execution: str) -> Optional[str]:
347         """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
348         flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),

```

```

349         so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
350         ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour.""
351         if not execution:
352             return None
353         try:
354             return self._client.execution_terminal_state(execution)
355         except Exception: # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
356             LOG.debug(
357                 "cloud-run: execution status probe failed for %s ? treating as unknown",
358                 execution,
359                 exc_info=True,
360             )
361             return None
362
363     def _handle_poll_timeout(
364         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
365     ) -> dict:
366         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
367
368         The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
369         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
370         ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
371         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
372         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
373         execution name so the operator can verify the state in the GCP console. A cancel
failure
374         NEVER masks the original timeout."""
375         try:
376             late = self._read_marker(done_uri)
377         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
378             LOG.warning(
379                 "cloud-run: final post-timeout marker check failed ? treating as
absent",
380                 exc_info=True,
381             )
382             late = None
383         if late is not None:
384             LOG.warning(
385                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
386                 "? rescuing the completed result",
387                 self._job_name,
388                 execution,
389             )
390             return late
391         try:
392             self._client.cancel_execution(execution)
393         except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
394             cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
395             LOG.warning(
396                 "cloud-run: best-effort cancel of execution=%s failed: %s",
397                 execution,
398                 cancel_exc,
399                 exc_info=True,
400             )
401         else:

```

```

402         cancel_note = "cancel requested"
403         LOG.info("cloud-run: requested cancel of execution=%s", execution)
404         raise PolicyTimeout(
405             f"{timeout_exc} [cloud-run job={self._job_name}] "
406             f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
407             "state in the GCP console]"
408         ) from timeout_exc
409
410     # --- storage-backed completion signal (bucket-poll) ---
411     def _read_marker(self, done_uri: str) -> Optional[dict]:
412         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
413         ref = parse_uri(done_uri)
414         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
415             return None
416         return (
417             self._read_json(done_uri) or {}
418         ) # empty dict still terminates the poll (present == done)
419
420     def _read_json(self, uri: str) -> dict:
421         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
422         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
423         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
424         try:
425             ref = parse_uri(uri)
426             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
427                 dst = os.path.join(td, ref.name or "obj.json")
428                 local = fetch(uri, dst, config=self._storage_config)
429                 with open(local, "r", encoding="utf-8") as f:
430                     data: Any = json.load(f)
431                 return data if isinstance(data, dict) else {}
432         except Exception:
433             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
434             return {}
435

```

8. assistant (2026-07-07T15:48:28.672Z)

Now let me read the supporting files: base.py, worker/__main__.py dispatch handler, execution_policy.py, and check orchestration.py for the caller.

9. user (2026-07-07T15:48:30.179Z)

```

1     """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3     COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4     (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5     core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6     (cloud-serving) ? without the core ever branching on the profile (D1).
7
8     The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9     non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod

```

```

16     from dataclasses import dataclass, field
17     from typing import Callable, Optional
18
19
20     class RemoteExecutionFailed(RuntimeError):
21         """A dispatched remote job's execution reached a TERMINAL state without ever writing
its
22         completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image
argparse-rejecting
23         an unknown flag) before the marker push, or the marker upload itself failed on an
otherwise
24         Succeeded run.
25
26         Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL
be
27         running): this says the execution is DEFINITELY over and no marker will ever
appear, so the
28         dispatcher must stop bucket-polling immediately instead of burning the full
29         ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message
carries the
30         job + execution name + terminal state + the ``gcloud`` command to read the execution
log, so
31         the whole "worker crashed before the done-marker" class fails fast AND
diagnostically."""
32
33
34     @dataclass(frozen=True)
35     class ComputeJobSpec:
36         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
37         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
38         deterministic regardless of where the job ran."""
39
40         video_uri: str # input (gs://? for cloud; a path/local:// for local)
41         out_uri: str # output ROOT (gs://? bucket for cloud)
42         segmenter: Optional[str] = None
43         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
44         config_overrides: tuple[str, ...] = ()
45         # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving
control-plane) is
46         # the authoritative validation gate: it runs the SAME validators with ITS resolved
config and only
47         # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video
decode, and
48         # (c) outright WRONG when the worker's config resolves different thresholds ? it can
REJECT a clip
49         # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no
control-plane)
50         # validating exactly as before.
51         skip_validation: bool = False
52
53
54     @dataclass(frozen=True)
55     class ComputeResult:
56         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
57         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
58         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
59         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
60

```

```

61         returncode: int
62         outputs_uri: str = ""
63         video_id: str = ""
64         report: dict = field(default_factory=dict)
65         # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
66         # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
67         # executions describe <execution>). Empty for the in-process path / backends without
one.
68         execution: str = ""
69
70
71     class ComputeBackend(ABC):
72         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
73         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
74         execution identically."""
75
76         @abstractmethod
77         def run_infer(
78             self,
79             spec: ComputeJobSpec,
80             *,
81             on_wait: "Callable[[float], None] | None" = None,
82         ) -> ComputeResult:
83             """Run ``spec`` to completion and return its ``ComputeResult``.
84
85             ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
86             still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
87             long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
88

```

10. user (2026-07-07T15:48:31.181Z)

```

140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF

```

```

path a true
162     no-op of work, not just of output."""
163     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164     cuda = (
165         probe_cuda_available() if profile else None
166     ) # torch-free on the default-OFF path
167     try:
168         enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169         return True
170     except StartupCheckError:
171         return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181         * rc 4 ? the bucket poll exhausted its budget (`PolicyTimeout`): the execution
may still be
182         running (the shim best-effort cancels it).
183         * rc 5 ? the execution has TERMINATED without a marker
(`RemoteExecutionFailed`): a worker
184         crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185         ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,
191             out_uri=args.out_uri,
192             segmenter=args.segmenter,
193             config_overrides=tuple(args.set or ()),
194             skip_validation=bool(getattr(args, "skip_validation", False)),
195         )
196         try:
197             result = compute.run_infer(spec)
198         except RemoteExecutionFailed as e:
199             logger.error(
200                 "[worker] remote execution terminated without a completion marker: %s", e
201             )
202             return 5
203         except PolicyTimeout as e:
204             logger.warning(
205                 "[worker] remote compute did not complete (no marker within budget): %s", e
206             )
207             return 4
208         logger.info(
209             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210             result.video_id,
211             result.returncode,
212             result.outputs_uri,
213         )
214         return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,

```



```

219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
227         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
228         try:
229             marker = {
230                 "video_id": video_id,
231                 "returncode": returncode,
232                 "segment_count": segment_count,
233                 "status": status,
234             }
235             local = os.path.join(scratch, "_done.json")
236             with open(local, "w", encoding="utf-8") as f:
237                 json.dump(marker, f)
238             storage.put(local, done_uri, config=storage_cfg)
239             logger.info("[worker] wrote completion marker -> %s", done_uri)
240         except Exception as e: # never fail a good run because the marker push hiccuped
241             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
242
243
244     def cmd_infer(args: argparse.Namespace) -> int:
245         config = load_config(args.config) if args.config else load_config()
246         _apply_overrides(config, args.set)
247         if not _run_startup_checks(config):
248             return 2
249
250         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
251         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254         compute = get_compute_backend(config)
255         if compute is not None:
256             return _dispatch_remote(compute, args)
257
258         storage_cfg = config.storage.model_dump()
259

```

11. user (2026-07-07T15:48:31.746Z)

```

1         """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3         Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4         - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7         - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10        **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real

```

```

11     waiting (a fake clock advances instantly).
12
13     **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14     transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15     ``execution.policy`` logger to watch the poll trail.
16     ""
17
18     from __future__ import annotations
19
20     import logging
21     import threading
22     import time as _time
23     from dataclasses import asdict, dataclass
24     from enum import Enum
25     from typing import Callable, Optional, TypeVar
26
27     T = TypeVar("T")
28
29     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
30     LOG = logging.getLogger("execution.policy")
31
32
33     class WaitMode(str, Enum):
34         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
35
36         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
37         ABORT = "abort" # one attempt; on hang/fail, stop immediately
38         BLOCK = "block" # bounded in-process blocking wait (no reschedule)
39
40
41     class PolicyTimeout(TimeoutError):
42         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
43
44
45     @dataclass(frozen=True)
46     class ExecutionPolicy:
47         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
48
49         mode: WaitMode = WaitMode.WAIT_AND_RESUME
50         poll_every_n_sec: float = 10.0
51         op_timeout_sec: float = (
52             120.0 # HARD per-attempt deadline ? the hang-killer that was missing
53         )
54         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
55         max_retries: int = 5 # retries (=> max_retries+1 attempts) for failures/hangs
56         backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
57         on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by P2/P3)
58         resumable: bool = True
59
60         def to_dict(self) -> dict:
61             d = asdict(self)
62             d["mode"] = self.mode.value
63             return d
64
65         @classmethod
66         def from_dict(cls, d: dict) -> "ExecutionPolicy":
67             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
68             kw = {k: v for k, v in d.items() if k in known}

```

```

69         if "mode" in kw and not isinstance(kw["mode"], WaitMode):
70             kw["mode"] = WaitMode(kw["mode"])
71         return cls(**kw)
72
73
74     #: The sane default (also what a job gets if it declares no policy).
75     DEFAULT_POLICY = ExecutionPolicy()
76
77
78     def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
79         """Deterministic exponential backoff (no jitter ? reproducible tests)."""
80         return policy.backoff_base_sec * (2**attempt)
81
82
83     def run_bounded(
84         fn: Callable[[], T],
85         policy: ExecutionPolicy = DEFAULT_POLICY,
86         *,
87         label: str = "op",
88         logger: Optional[logging.Logger] = None,
89         sleep: Callable[[float], None] = _time.sleep,
90     ) -> T:
91         """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
92         backoff.
93
94         Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
95         hung, or re-raises the
96         last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
97         retries).
98
99         """
100         log = logger or LOG
101         attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
102         last_exc: Optional[BaseException] = None
103         for attempt in range(attempts):
104             box: dict = {}
105             done = threading.Event()
106
107             # box/done bound as defaults (not closed over) ? the thread is started + joined
108             # same iteration, but explicit binding keeps each attempt isolated and silences
109             B023.
110
111             def _runner(_box: dict = box, _done: threading.Event = done) -> None:
112                 try:
113                     _box["val"] = fn()
114                 except (
115                     BaseException
116                 ) as e: # relay ANY error (incl. timeouts) to the caller thread
117                     _box["exc"] = e
118                 finally:
119                     _done.set()
120
121             threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
122             if done.wait(timeout=policy.op_timeout_sec):
123                 if "exc" in box:
124                     last_exc = box["exc"]
125                     log.info(
126                         "%s: attempt %d/%d failed: %s",
127                         label,
128                         attempt + 1,
129                         attempts,
130                         last_exc,
131                     )
132                 else:
133                     if attempt:
134                         log.info(

```

```

129         "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
130     )
131     return box["val"] # type: ignore[return-value]
132 else:
133     last_exc = PolicyTimeout(
134         f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
135     )
136     log.warning(
137         "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
138         label,
139         attempt + 1,
140         attempts,
141         policy.op_timeout_sec,
142     )
143     if attempt < attempts - 1:
144         wait = _backoff_sec(policy, attempt)
145         log.debug("%s: backing off %.1fs before retry", label, wait)
146         sleep(wait)
147     assert last_exc is not None
148     log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
149     raise last_exc
150
151
152 def poll_until(
153     check: Callable[[], Optional[T]],
154     policy: ExecutionPolicy = DEFAULT_POLICY,
155     *,
156     label: str = "op",
157     logger: Optional[logging.Logger] = None,
158     sleep: Callable[[float], None] = _time.sleep,
159     monotonic: Callable[[], float] = _time.monotonic,
160     on_tick: Optional[Callable[[float], None]] = None,
161 ) -> T:
162     """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
163     value, or raise
164     ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
165     still in progress.
166     Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
167     INFO/WARNING.
168     ``on_tick(elapsed_sec)`` (optional) fires once per still-waiting iteration ? the
169     caller's live
170     progress heartbeat (#482). Best-effort by contract: a raising callback is swallowed
171     (DEBUG-logged), never allowed to kill the wait it reports on.
172     """
173     log = logger or LOG
174     start = monotonic()
175     polls = 0
176     log.info(
177         "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
178         label,
179         policy.poll_every_n_sec,
180         policy.max_wait_sec,
181     )
182     while True:
183         result = check()
184         if result is not None:
185             log.info(
186                 "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
187             )
188             return result
189         elapsed = monotonic() - start
190         if elapsed >= policy.max_wait_sec:
191             log.warning(
192                 "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed

```

```

190         )
191         raise PolicyTimeout(
192             f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
193         )
194     polls += 1
195     log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
196     if on_tick is not None:
197         try:
198             on_tick(elapsed)
199         except Exception: # noqa: BLE001 ? progress reporting must never kill the
wait
200             log.debug("%s: on_tick callback failed", label, exc_info=True)
201     sleep(policy.poll_every_n_sec)
202

```

12. assistant (2026-07-07T15:49:26.377Z)

Let me examine the actual control-plane caller (orchestration.py) and the parser to understand the real motivating path and whether skip_validation is wired.

13. user (2026-07-07T15:49:31.046Z)

```

411: from backend.compute import ComputeJobSpec, get_compute_backend
577: spec = ComputeJobSpec(
582:     skip_validation=True,
589:     result = compute.run_infer(

```

14. user (2026-07-07T15:49:31.060Z)

```

194:     skip_validation=bool(getattr(args, "skip_validation", False)),
280: # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
282: if getattr(args, "skip_validation", False):
284:     "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
754:     "--skip-validation",

```

15. assistant (2026-07-07T15:49:35.362Z)

Let me read the orchestration control-plane dispatch path in full.

16. user (2026-07-07T15:49:35.495Z)

```

560     wasb_cfg = getattr(idx_cfg, "wasb", None)
561     if wasb_cfg is not None:
562         decode_backend = getattr(wasb_cfg, "decode_backend", None)
563         if decode_backend:
564             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565         batch_size = getattr(wasb_cfg, "batch_size", None)
566         if batch_size:
567             overrides.append(f"indexing.wasb.batch_size={batch_size}")
568         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569         if prefetch:
570             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571     # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
572     # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
573     # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
574     # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
575     # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,

```

```

576     # then worker-rejected at its default 20.0 after a successful GPU dispatch).
577     spec = ComputeJobSpec(
578         video_uri=req.video_path,
579         out_uri=out_root,
580         segmenter=seg_name,
581         config_overrides=tuple(overrides),
582         skip_validation=True,
583     )
584     try:
585         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
586         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
587         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
588         # also gives the SPA a stall signal to time out on, instead of a blind wall
589         result = compute.run_infer(
590             spec,
591             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
592         )
593     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
594         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
595         return _failed(f"cloud dispatch error: {e}", True)
596     if result.returncode != 0:
597         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
598     if result.video_id and result.video_id != video_id:
599         # The worker re-hashes the bucket bytes; a divergence means the input changed
600         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
601         # diagnosable.
602         logger.warning(
603             "[API] cloud video_id %s != local %s ? input bytes differ between the API
604             hash "
605             "and the worker hash; ingesting under the local id.",
606             result.video_id,
607             video_id,
608         )
609     notify("ingesting (cloud_run)")
610     storage_dict = (
611         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
612         if hasattr(storage_cfg, "model_dump")
613         else {}
614     )
615     try:
616         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
617     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
618         # 500 or leak rows
619         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
620         return _failed(
621             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
622             True,
623         )
624     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
625     # drops the old ones.
626     segments = [
627         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
628     ]
629     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
630     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
631     # is the real
632     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
633     # <execution>).

```

```

630     provenance = {
631         "path": "cloud_run",
632         "requested": req.compute,
633         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
634         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
635         "execution": result.execution,
636         "outputs_uri": result.outputs_uri,
637     }
638     logger.info(
639         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
640         result.execution,
641         provenance["job"],
642         provenance["region"],
643         result.outputs_uri,
644         video_id,
645     )
646     return (
647         {
648             "video_id": video_id,
649             "status": "success",
650             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
651             "results": [r.model_dump() for r in val_results],
652             "play_segments": segments,
653             "compute": provenance,
654         },
655         True,
656     )
657
658
659 def _execute_processing(
660     config=None, db=None, req=None, progress=None
661 ) -> Dict[str, Any]:
662     """The full hash ? validate ? segment ? report pipeline for one video.
663
664     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
665     module-level ``global_config``/``db_instance`` (backward-compatible with existing
666     tests that pass only ``req``). Route handlers always pass explicit params; older
667     test code that calls ``_execute_processing(req)`` still works.
668
669     ``progress`` is an optional callable(stage: str) used to surface coarse job
670 progress.
671     Raises on unexpected internal errors ? the caller records those as a job failure
672 """
673     import backend.main as _main # call-time seam: main's globals/model stay patchable
674
675     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
676     # shift positional args ? req arrives in position 0 (config), progress in position 1
677     (db).
678     if config is not None and not isinstance(config, AppConfig):
679         # Old signature: _execute_processing(req, progress=None)
680         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
681             req = db

```

17. assistant (2026-07-07T15:51:16.345Z)

I now have a complete picture. Let me verify the two halves of the motivating scenario are covered and confirm the control-plane path (the real production motivator) handles the new exception, then check there's an existing orchestration dispatch-error test.

18. user (2026-07-07T15:51:19.051Z)

Found 4 files
tests/test_compute_backend.py

```
tests\test_api_cloud_dispatch.py
tests\test_execution_policy.py
tests\test_main_module_budget.py
```

19. user (2026-07-07T15:51:19.551Z)

```
744         "Default-OFF: omit it for the normal local run.",
745     )
746     pi.add_argument(
747         "--set",
748         action="append",
749         default=[],
750         metavar="k=v",
751         help="config override, e.g. --set indexing.skip_ai_handoff=true",
752     )
753     pi.add_argument(
754         "--skip-validation",
755         action="store_true",
756         help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
757         "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
758         "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
759         "`worker infer` CLI still validates.",
760     )
761     pi.set_defaults(func=cmd_infer)
762
763     pt = sub.add_parser(
```

20. assistant (2026-07-07T15:51:23.732Z)

Let me check the orchestration control-plane dispatch-error test to confirm the real motivating path is covered.

21. user (2026-07-07T15:51:26.253Z)

```
92:         raise AssertionError(
146: def test_cloud_dispatch_ingests_segments(cloud_env):
165: def test_cloud_dispatch_builds_correct_spec(cloud_env):
189: def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
202: def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
222: def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
239: def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
255: def test_cloud_failure_marks_failed_no_rows(cloud_env):
275: def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
289: def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
330: def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
367: def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
monkeypatch):
384: def test_default_off_runs_in_process(tmp_path, monkeypatch):
425: def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
454: def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
493: def test_per_request_local_forces_in_process(cloud_env):
516: def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
558: def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
586: def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
605: def test_capabilities_reports_cloud_available(cloud_env):
617: def test_capabilities_cloud_unavailable_by_default(monkeypatch, tmp_path):
630: def test_full_api_process_to_query_via_cloud(cloud_env):
658: def test_local_override_survives_api_boundary(cloud_env):
691: def test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypatch):
743: def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
765: def test_local_override_stamps_in_process_provenance(cloud_env):
```



```

781: def test_default_in_process_stamps_provenance(tmp_path, monkeypatch):
802: def test_cloud_requested_unconfigured_stamps_not_dispatched(tmp_path, monkeypatch):
839: def test_validation_failure_stamps_not_run_provenance(tmp_path, monkeypatch):
857: def test_full_api_process_job_result_carries_cloud_provenance(cloud_env):
883: def test_in_process_detection_runs_under_the_local_lane(cloud_env):
918: def test_wait_progress_message_is_honest_and_minute_granular():

```

22. user (2026-07-07T15:51:34.693Z)

```

239     def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
240         """The control-plane is the AUTHORITATIVE validation gate: it validated + only
dispatches on PASS
241         (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on
the worker is
242         redundant, decodes the video a second time, and ? because the control-plane's
validation config
243         isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07:
passed at
244         validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
245         carry --skip-validation, telling the worker to trust the gate. Regression guard for
that wiring."""
246         _db, _bucket, monkeypatch = cloud_env
247         fake = _FakeCloudClient()
248         _inject_fake(monkeypatch, fake)
249         m._execute_processing(
250             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
251         )
252         assert "--skip-validation" in fake.calls[0]
253
254
255     def test_cloud_failure_marks_failed_no_rows(cloud_env):
256         db, _bucket, monkeypatch = cloud_env
257         # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
258         db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
259         fake = _FakeCloudClient(returncode=1)
260         _inject_fake(monkeypatch, fake)
261
262         out = m._execute_processing(
263             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
264         )
265         assert out["status"] == "failed"
266         assert "returncode 1" in out["message"]
267         assert out["play_segments"] == []
268         assert db.query_play_segments(out["video_id"], {}) == []
269         # It DID dispatch + run remotely (then failed rc=1), so the path is honestly
cloud_run + dispatched.
270         assert (
271             out["compute"]["path"] == "cloud_run" and out["compute"]["dispatched"] is True
272         )
273
274
275     def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
276         _db, _bucket, monkeypatch = cloud_env
277         fake = _FakeCloudClient()
278         _inject_fake(monkeypatch, fake)
279         local_path = str(tmp_path / "local_only.mp4")
280
281         out = m._execute_processing(
282             m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
283         )
284         assert out["status"] == "failed"
285         assert "cloud input" in out["message"]
286         assert fake.calls == [] # never dispatched a local-only path

```

287
288

23. assistant (2026-07-07T15:52:09.986Z)

I now have everything I need to judge test adequacy. Let me record my conclusion.

Analysis summary

****The fast-fail mechanism is thoroughly and non-tautologically tested.**** Walking each new test in `tests/test_compute_backend.py`:

- `test_run_infer_fails_fast_when_execution_terminal_without_marker` (L469)` drives the real path `poll_until ? _poll_check ? _terminal_execution_state ? raise``. It is a genuine regression detector, not tautological: `_CrashedNoMarkerClient`` carries an internal `assert status_polls <= 5`` anti-spin guard (L144) AND the policy bounds `max_wait_sec=2.0``, so a regression that silently fell back to marker-only polling would either trip the guard or surface `PolicyTimeout`` (not `RemoteExecutionFailed``) ? either way the test fails. The `status_polls == 1`` assertion proves the poll aborted on the first terminal observation.
- `test_dispatch_remote_maps_terminal_without_marker_to_rc5` (L493)` proves the `_dispatch_remote`` rc-5 mapping (and `rc != 4``, distinct from the timeout path).
- `test_status_probe_error_is_swallowed_and_falls_back_to_marker_polling` (L561)` proves a raising probe yields `PolicyTimeout``, not a manufactured `RemoteExecutionFailed`` ? a real behavioral distinction (an un-swallowed raise would propagate `RuntimeError``).
- `test_present_marker_wins_over_terminal_status` (L540) +`
`test_terminal_status_with_marker_written_in_race_resolves_via_marker` (L605)` cover marker-authority and the one-shot race re-check (`exists_calls == 2``).
- `test_succeeded_state_without_marker_also_fails_fast``,
`test_default_execution_terminal_state_is_none``,
`test_real_client_terminal_state_none_for_unresolvable_name``,
`test_terminal_execution_state_none_for_empty_execution`` cover the generalization, the default-OFF seam, and the empty/unresolvable-name guards.

****The motivating scenario is covered at the correct abstraction boundary.**** A fake client cannot run a real old image, so the `argparse-abort` is (correctly) simulated as its only observable consequence: `*terminal execution + no marker* ?` which the fast-fail is proven to convert to `RemoteExecutionFailed ? rc 5``. The `skip_validation ? argv`` half is separately and adequately covered by `test_dispatch_argv_carries_skip_validation_when_set` (L272)` and, at the control-plane level, `test_api_cloud_dispatch.py::test_cloud_dispatch_skips_worker_revalidation` (L239)`. The real production motivator (control plane ? orchestration `_maybe_dispatch_remote``, L577-595) catches `RemoteExecutionFailed`` via a broad `except Exception ? _failed``, so the exception is handled there too (not turned into rc 5 ? rc 5 is the worker-CLI path only).

I considered the one candidate the lens suggests ? an integration test tying `skip_validation`` dispatch to the rc-5 fast-fail ? and judge it ****defense-in-depth, not a defect-grade gap****: the two halves are code-independent (`argv` construction vs. `poll/status` probe) with no shared state, and each is separately covered, so no plausible single regression is left uncaught by their separation. Reporting it would be padding, which the brief warns against.

24. user (2026-07-07T15:52:19.083Z)

Structured output provided successfully